

Sistema de videojuego interactivo

Maria Alejandra Posada - Maria Camila Garcia Alvarez - Sebastian Villa
Electrónica digital II

Índice

1. Identificación del proyecto	2
2. Resumen	2
3. Descripción del hardware	2
4. Descripción del software	3
5. Estructura del código	3
6. Explicación de funciones principales	3
7. Procedimiento de prueba	4
8. Manejo de errores y seguridad	4
9. Conclusión	5
10. Referencias	5

1. Identificación del proyecto

- **Nombre del proyecto:** Sistema de videojuego interactivo con interfaz gráfica en pantalla OLED
- **Integrantes del equipo:** Maria Alejandra Posada - Maria Camila Garcia Alvarez - Sebastian Villa

2. Resumen

el siguiente proyecto desarrolla un videojuego interactivo en el microcontrolador ESP32, utilizando una pantalla OLED como interfaz gráfica y botones para el control del usuario. El objetivo del juego es esquivar obstáculos generados aleatoriamente mientras se incrementa el puntaje. El sistema incluye diferentes modos de juego, aumento progresivo de la dificultad y diversos estados de funcionamiento como menú, juego, pausa y fin de partida. Además, se integra retroalimentación visual y sonora mediante gráficos en pantalla y un buzzer.

3. Descripción del hardware

- **ESP32:** Microcontrolador principal encargado de la adquisición, procesamiento y control del sistema.
- **Pantalla OLED (I2C)** Dispositivo de visualización para la interfaz gráfica
- **Botones** Pines 12,14 y 27: Permiten al usuario interactuar con el sistema, desplazarse en el menú y controlar el movimiento de la nave.
- **Buzzer** pin 25: Dispositivo de salida sonora que proporciona retroalimentación auditiva durante la interacción del usuario

Diagrama de conexión:

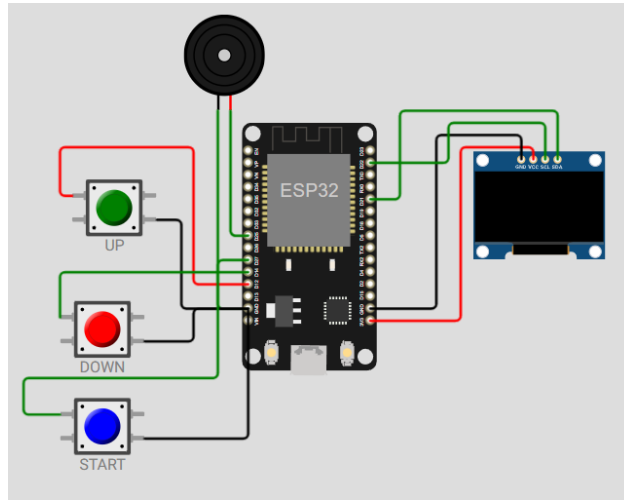


Figura 1: Diagrama

4. Descripción del software

- **Lenguaje usado:** MicroPython
- **Librerías:** machine, utime, urandom, ssd1306
- **IDE:** Thonny

5. Estructura del código

- Archivo principal: `main.py`
- No se usan módulos adicionales.
- La lógica principal se ejecuta dentro de un bucle `while True`, en el cual se implementa una máquina de estados que gestiona los modos de funcionamiento: menú, juego, pausa y fin de partida.

6. Explicación de funciones principales

- **sonar(freq, dur):** Esta función controla el buzzer mediante señal PWM, permitiendo generar sonidos con una frecuencia y duración determinadas. Se utiliza como retroalimentación auditiva en eventos como selección de opciones o colisiones.
- **reset juego():** Reinicia las variables principales del juego, como la posición de la nave, el puntaje, la lista de obstáculos y la dificultad. Se ejecuta al iniciar una nueva partida.

- **dibujar nave(x, y):** Se encarga de representar gráficamente la nave del jugador en la pantalla OLED mediante el uso de píxeles y figuras simples.
- **dibujar nube(x, y):** Dibuja los obstáculos del juego en forma de nubes, utilizando primitivas gráficas básicas en la pantalla OLED.
- **dibujar estrellas():** Genera y actualiza un fondo animado de estrellas, desplazándolas horizontalmente para simular movimiento y mejorar la estética visual del juego.
- **Bucle principal (while True):** Contiene la lógica central del sistema. Dentro de este ciclo se gestiona la máquina de estados del juego, se procesan las entradas de los botones, se actualiza la posición de los elementos, se detectan colisiones, se controla el puntaje y se actualiza la pantalla en tiempo real.

7. Procedimiento de prueba

- **Verificación de botones:** Se comprueba el correcto funcionamiento de los botones (UP, DOWN y START), validando la navegación en el menú y el control del movimiento de la nave.
- **Interacción con el menú:** Se seleccionan los diferentes modos de juego, verificando el desplazamiento entre opciones y la correcta selección del modo.
- **Ejecución del juego:** Se inicia una partida y se observa el movimiento de la nave, la generación de obstáculos y la visualización del puntaje en tiempo real.
- **Detección de colisiones:** Se provoca el contacto entre la nave y los obstáculos para verificar la correcta detección de colisiones y la transición al estado de fin de juego.
- **Incremento de dificultad:** Se evalúa el aumento progresivo de la dificultad conforme incrementa el puntaje, verificando mayor velocidad o frecuencia de obstáculos.
- **Función de pausa:** Se activa y desactiva la pausa durante el juego, comprobando que el sistema detiene y reanuda correctamente la ejecución.
- **Reinicio del juego:** Se valida el retorno al menú principal después del estado de game over, verificando el reinicio adecuado de las variables del sistema.

8. Manejo de errores y seguridad

El sistema implementa diferentes mecanismos que garantizan un funcionamiento estable y seguro:

- **Control de límites de movimiento:** La posición de la nave se restringe dentro de los límites de la pantalla, evitando valores fuera de rango que puedan generar errores de visualización.

- **Gestión de obstáculos:** Los obstáculos se almacenan en una lista y se eliminan cuando salen de la pantalla, evitando el crecimiento innecesario de memoria.
- **Control de entradas (anti-rebote):** Se utilizan pequeños retardos después de la lectura de los botones para evitar múltiples detecciones no deseadas por efecto rebote.
- **Control de colisiones:** La detección de colisiones se realiza dentro de rangos definidos, evitando errores lógicos en la interacción entre la nave y los obstáculos.

9. Conclusión

En este laboratorio se desarrolló un videojuego interactivo utilizando el microcontrolador ESP32, integrando el uso de una pantalla OLED, botones de control y un buzzer. Se implementó una estructura basada en máquina de estados que permitió gestionar de forma organizada los diferentes modos de funcionamiento del sistema. El proyecto permitió aplicar conceptos de programación en sistemas embebidos, manejo de entradas y salidas digitales, generación de gráficos y control en tiempo real. Además, se logró una interacción fluida con el usuario mediante retroalimentación visual y sonora.

10. Referencias

1. MicroPython Documentation (2024). machine module.
2. Espressif Systems. ESP32 Technical Reference Manual.
3. Documentación de la librería SSD1306 para pantallas OLED.
4. Wokwi Simulator Documentation.
5. Monk, S. Programming the ESP32 in MicroPython.